

重复词样本递归递归递归

这份笔记刻意让递归”一词反复出现用来测试选区定位是否真的命中指定的那一处而不是偷偷退回到第一次出现的位置递归不是关键词密度的练习但重复词样本必须如此构造每次出现都该是一句完整可读的话。

第一处递归出现之后我们立刻引入第二处递归递归的可靠性来自归纳而不是来自祈祷当递归调用的参数严格向基线条件收敛时递归是可证明的当递归参数原地踏步时递归就是死循环的另一种写法。

第三处递归出现在这里调试递归的技巧是打印缩进进入函数时输出一段前缀加参数返回时输出结果肉眼观察递归的进出节奏比在调试器里单步两百次有效得多递归的调用树一旦可视化理解就完成了一半。

中段重复词的分布

到目前为止递归出现了不止五次分布在不同段落不同页面选区系统若要支持第二处第五处第八处这样的定位语义就必须在这些重复出现之间做出准确区分本页之后还有更多递归在等待。

递归与迭代的互相改写是有边界的任何尾递归都能机械地改写成循环但分治型递归如归并排序需要显式栈辅助改写之后可读性往往下降递归在这里的价值是表达力不是性能性能敏感的路径可以先用递归想清楚再用迭代落地。

继续计数这是又一个自然段里的递归树遍历解析正则引擎数据库查询计划工业界代码里递归的身影远比课堂练习里多递归并不幼稚幼稚的是不加终止检查的递归每一段都在把同一个词放进不同的语境这正是重复词样本的构造方式。

再补一段方法论阅读陌生的递归函数时先找基线条件再找分解点最后才看返回值如何组合三步走完函数的行为就定型了递归的阅读顺序与书写顺序恰好相反写的时候先想分解读的时候先找出口。

补充一个观察团队里最常被争论的不是要不要用递归而是递归的深度预算给多少解析用户输入的递归预算通常是一百层遍历受控数据结构的递归预算可以放宽到一万层预算定下来之后护栏就该出现在代码里而不是出现在事后复盘里。递归配上预算才算是工程决策。

重复词样本的构造说明也顺带记录在此至少出现八次位置分布在标题段首段中与段尾每一处都必须是完整句子的一部分递归这个词在这里的密度是刻

意的换到真实笔记里正常密度大约是每千字一次。

尾段接近样本的末尾

第八处递归大约应该出现在这一页附近为了让重复词样本更接近真实材料这里的每句话仍然是完整有信息量的句子而不是把同一个词机械堆在一起递归的学习应当顺着这条线走下去先信归纳再读调用栈最后写一个属于自己的递归小程序。

从工程视角再看一眼递归任何递归实现都应当配一个深度上限或规模护栏尤其是处理外部输入的解析器递归下降解析器遇到恶意深嵌套的输入时没有护栏的递归会把进程直接打穿递归优雅防护也不能少。

最后再安排两处递归:一句在段首本句)一句在段尾样本到此收束递归的讨论将在长文样本里继续选区断言若全部命中说明重复词定位的语义已经闭合。